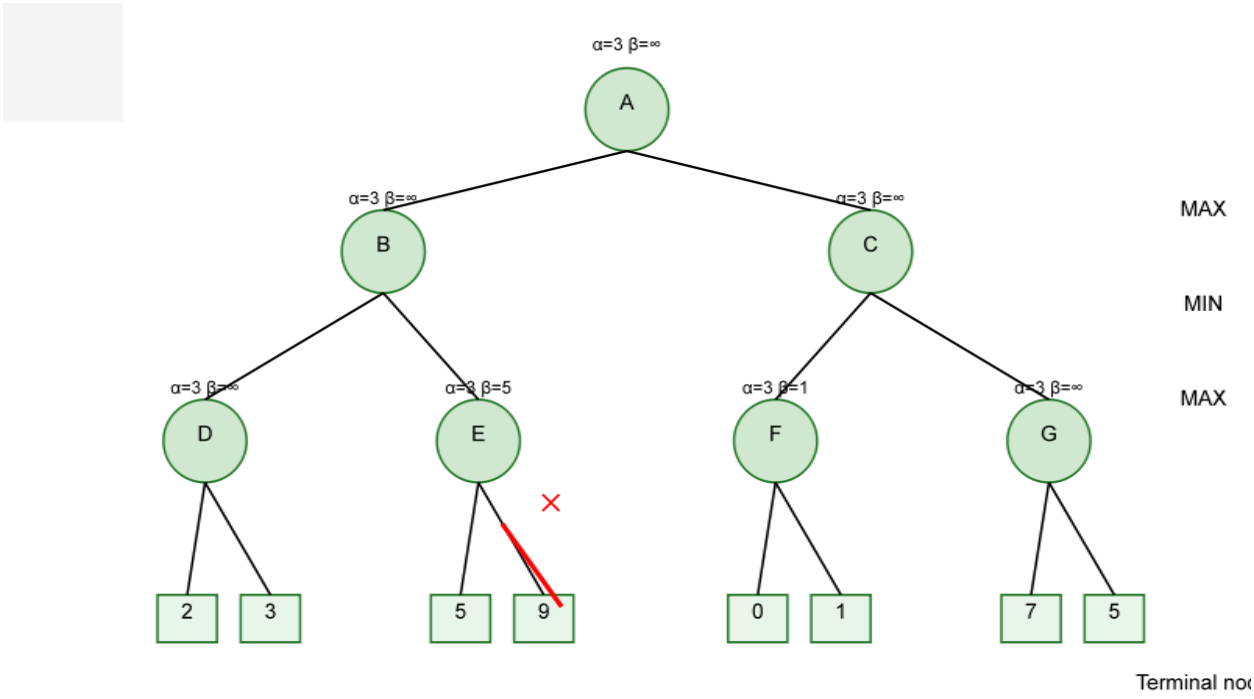
AI Lab-2
BS DS FALL 2024

TASK 01: Minimax Algorithm



Introduction to Minimax Algorithm:

The **Minimax algorithm** is used in two-player zero-sum games where:

Maximizer tries to maximize the score
Minimizer tries to minimize the score
The algorithm explores the game tree in **depth-first order**
Terminal states are evaluated using a utility function
Non-terminal states (when the depth limit is reached) are evaluated using a heuristic

Code Template:

```
class Minimax:
  def __init__(self, game_state, max_depth=9):
      self.game_state=game_state
      self.max_depth=max_depth
      self.nodes_expanded=0  #Track number of explored nodes

  def is_terminal(self, state):
        # Check if the game has reached a terminal state(win,lose,draw)
        pass
  def utility(self, state):
        # Return the utility value of the terminal state
      pass
  def heuristic(self,state):
        # Heuristic function for evaluating non-terminal states
       pass
  def minimax(self, state, depth, maximizing_player):
        # Implement the Minimax algorithm
       pass

  def best_move(self, state):
        # Determine the best move using Minimax
       pass
```

Lab Tasks:

1. Implement the Minimax algorithm for a two-player game (e.g., Tic-Tac-Toe).
2. Test the algorithm for different board configurations.

Students must:

Run Minimax with:

- Depth = 9 (Full game search)
- Depth = 4
- Depth = 2

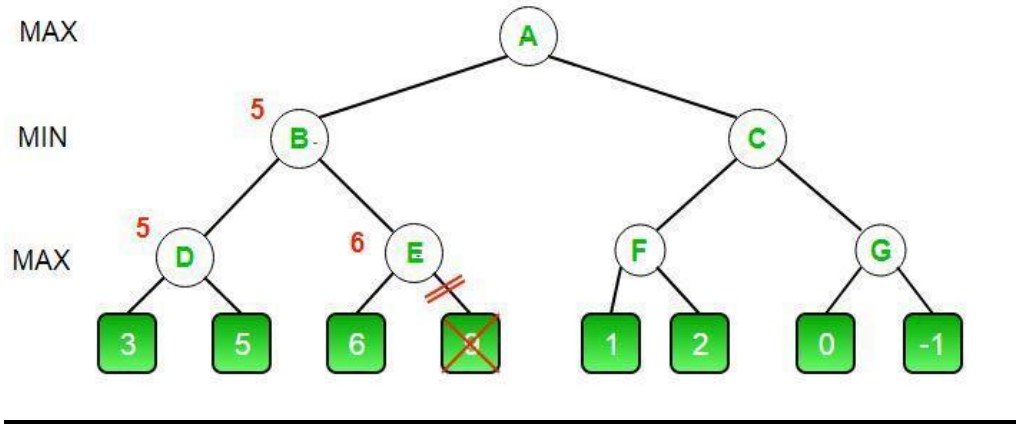
Record:

- Best move selected
- Nodes expanded
- Time taken

Compare how reducing depth affects:

- Performance
- Optimality

Task 02: Alpha-Beta Pruning



Introduction to Alpha-Beta Pruning:

Alpha-Beta Pruning improves Minimax by eliminating unnecessary branches.

Important Variables

- **Alpha (α):**
  Best value maximizer can guarantee so far
- **Beta (β):**
  Best value minimizer can guarantee so far

If at any point:

`alpha >= beta`

We stop exploring that branch (PRUNE).

Problem:

The Game of Tic-Tac-Toe: In this task, we will apply Alpha-Beta Pruning to optimize the Minimax algorithm for a game of Tic-Tac-Toe. The objective is to make the AI play optimally while reducing the number of nodes it evaluates to make its move.

Code Template:

```
class AlphaBetaPruning:
    def __init__(self, depth=9, game_state, player):
        self.max_depth = depth
        self.game_state = game_state
        self.player = player
        self.nodes_expanded = 0  # Track DFS nodes visited

    def is_terminal(self, state):
        # Check if the game has reached a terminal state (win, lose, draw)
        pass

    def heuristic(self,state):
         # Heuristic function for evaluating non-terminal states
         pass

    def utility(self, state):
        # Return the utility value of the terminal state
        pass

    def alphabeta(self, state, depth, alpha, beta, maximizing_player):
        # Implement Alpha-Beta pruning
        pass

    def best_move(self, state):
        # Determine the best move using Alpha-Beta pruning
        pass
```

Lab Tasks:

1. Implement the Alpha-Beta Pruning algorithm for the game of Tic-Tac-Toe.
2. Students are encouraged to implement their own heuristic evaluation functions to further improve the AI's decision-making.
3. Test the algorithm by playing against the AI at different difficulty levels (adjusting depth).
4. Measure the performance of the Alpha-Beta Pruning algorithm by counting the number of nodes evaluated for each move and compare it with the basic Minimax algorithm.

Main function and Running example:

```
def main():
    board = [' ' for _ in range(9)]
    human_player = 'X'
    ai_player = 'O'
    current_player = human_player

    ai = Minimax(board)  # or AlphaBetaPruning(depth, board, ai_player)

    while ' ' in board:
        print_board(board)

        if current_player == human_player:
            move = int(input(f"Enter your move (0-8): "))
            if board[move] == ' ':
                board[move] = human_player
                if check_winner(board, human_player):
                    print_board(board)
                    print("You win!")
                    return
                current_player = ai_player
            else:
                print("Invalid move, try again.")
        else:
            move = ai.best_move(board)
            board[move] = ai_player
            print(f"AI selects position {move}")
            if check_winner(board, ai_player):
                print_board(board)
                print("AI wins!")
                return
            current_player = human_player
    print_board(board)
    print("Game ended in a draw!")

def print_board(board):
    print(f"{board[0]} | {board[1]} | {board[2]}\n---------\n{board[3]} |
{board[4]} | {board[5]}\n---------\n{board[6]} | {board[7]} | {board[8]}")

def check_winner(board, player):
    win_conditions = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)]
    return any(all(board[pos] == player for pos in condition) for condition in
win_conditions)
main()
```